

User Story: User Story 6: Display Athlete BMI Value, Category and Max Heart Rate

Category would be a method described before this one, so it will not be included in the test cases for this display.

Method Description:

Displays the athlete name, BMI as a double. MHR is also displayed.

Pre	Post
Name = Caleb BMI = 23 MHR = 200	Caleb BMI: 23 MHR: 200
Name = Jaden BMI = 18.7 MHR = 201	Jaden BMI: 18.7 MHR: 201
Name = Candy BMI = 17.7 MHR = 190	Candy BMI: 17.7 MHR: 190

Method Signature: `public static void displayAthleteBMI(String[] nameBMI, double[] bmiDisplay, String[] categoryChosen, double[] mhrDisplay)`

Design: Put an image of your pseudocode handwritten on paper, white board or tablet for the algorithm.

Method Signature

```
for count = 0    count < nameBMI.length    ++count
```

```
    print "nameBMI[count]  
    bmiDisplay(count)  
    mhrDisplay(count)
```

User Story: **User Story 7: List all athletes outside normal BMI range**

Method Description:

Displays Athlete names and whether above or below normal

Above normal: Deb

Below normal: Monty

When none outside range display "No athletes outside of normal range"

Pre	Post
Name = Caleb BMI = 40 Name = Candy BMI = 17.7 Name = Lauren BMI = 25	Above normal: Caleb Below normal: Candy Above normal: Lauren
Name = Caleb BMI = 18.5 Name = Lauren BMI = 24.9	"No athletes outside of normal range"

Method Signature: **public static void outNormalBMI(String[] bmiNoNorm, String[] nameNoNorm)**

Design: Put an image of your pseudocode handwritten on paper, white board or tablet for the algorithm.

```
Method Signature
for count = 0 count < nameNoNorm.Length ++count
    if bmiNoNorm(count) > 24.9
        print "Above normal: nameNoNorm(count)"
    else if bmiNoNorm(count) < 18.5
        print "Below normal: nameNoNorm(count)"
```

User Story: **User Story 8: Calculate and Display Average of the Max Heart Rates**

Method Description:

Calculates the average Max Heart Rate of the team.

Pre	Post
MHR Array : [180, 120, 190, 10]	Average = 125
MHR Array : [20]	Average = 20
MHR Array : [180, 150, 120, 100, 110, 170]	Average = 138.333333
MHR Array : [300, 300]	Average = 300

Method Signature: **public static double calculateAverage (double[] MHR)**

Design: Put an image of your pseudocode handwritten on paper, white board or tablet for the algorithm.

```
double calculateAverage (double[] numbers) MHR
double average = 0
for (count = 0; count < numbers.length; ++count)
    average = (average + numbers[count])
return average
```

The handwritten pseudocode is written on a grid background. It defines a method `calculateAverage` that takes a `double[] numbers` array as input. It initializes a `double average` variable to 0. A `for` loop iterates from `count = 0` to `count < numbers.length`, incrementing `count` by 1 each time. Inside the loop, the `average` is updated as `average = (average + numbers[count])`. The final `average` is returned. There are some handwritten annotations: an arrow points from 'MHR' to the parameter 'numbers', and a note 'heart Rate, length' is written below the loop.

User Story: **User Story 9: Identify Athlete with Highest Max Heart Rate**

Method Description:

Determines the **index of the** highest Max Heart Rate out of the given list of athletes.

Pre	Post
MHR Array : [180, 120, 190, 10]	2
MHR Array : [20]	0
MHR Array : [180, 150, 120, 100, 110, 170]	0
MHR Array : [300, 300]	0

Method Signature: **public static int highestMHR(String[] nameHigh, double[] mhrHigh)**

Design: Put an image of your pseudocode handwritten on paper, white board or tablet for the algorithm.

```
Method Signature
highestMHR = 0
for count = 0 to nameHigh.length - 1
    if highestMHR < mhrHigh[count]
        highestMHR = mhrHigh[count]
        MHRindex = count
end for loop
```

A print will later be added to finish the entirety of this user story.

User Story: **User Story 10: List all athletes above or equal to Max Heart Rate average**

Method Description:

Displays names of athletes whose max heart rates are greater than or equal to the average MHR

Pre	Post
MHR Array : [180, 120, 190, 10]	Name Indexes: 0, 2
MHR Array : [20]	Name Indexes: 0
MHR Array : [180, 150, 120, 100, 110, 170]	Name Indexes: 0, 1, 5
MHR Array : [300, 300]	Name Indexes: 0, 1

Method Signature: **public static void displayAboveAverage(String[] nameAbove, double[] mhrAboveAvg, double averageCompare)**

Design: Put an image of your pseudocode handwritten on paper, white board or tablet for the algorithm.

```
Method Signature  
for count=0 count < nameAbove.length ++count  
  if mhrAboveAvg(count) ≥ averageCompare  
    print nameAbove(count)
```

It just displays the names not the bmi, although that could be added later.

User Story: User Story 11: Calculate and Display Training Heart Rate

Method Description:

When the trainer types in Y or y, they are allowed to input a non-zero, non-negative number to indicate training percentage. The MHR of each athlete is multiplied by that percentage, and then it is displayed next to their name. When the program ends, whether it be because the functions was completed, or the trainer chose N or n, display "Training Program Analysis complete"

Pre	Post
Continue? = Y Percentage = 85	All athletes MHR * 0.85
Continue? = y Percentage = 85	All athletes MHR * 0.85
Continue? = N	"Training Program Analysis complete"
Continue? = n	"Training Program Analysis complete"

Method Signature: `public static void trainingHeartRate(Scanner answerInput, String[] nameTrain, double[] mhrTrain)`

Design: Put an image of your pseudocode handwritten on paper, white board or tablet for the algorithm.

```
Method Signature
answer = answerInput.charAt(0)
while answer != 'Y', 'y', 'N', 'n'
    "Try again"

if answer == 'Y' or 'y'
    percentage = answerInput.nextInt()
    while percentage <= 0
        "Try again"

for count = 0; count < nameTrain.length; count++
    mhrTrain[count] = mhrTrain[count] * percentage
    print "nameTrain[count] and mhrTrain[count]"
```